CPSC 6440
Problem Set 2 Report

Anna Su

October 27, 2025

1 Graph Neural Networks for Single-Cell Analysis

1.1 Experiment with different values of k for graph construction and report how connectivity affects downstream performance.

As shown in Table 1, as the k value increases, the loss also increases while the test accuracy slightly decreases. This suggests that a lower k value may lead to better model performance in this context.

k value	Loss	Test Accuracy
3	0.07	0.97
5	0.11	0.97
10	0.18	0.97
15	0.23	0.96
25	0.39	0.94

Table 1: Effect of different k values on model performance

1.2 Visualize predicted vs. true expression for several genes by experimenting with different t values.

As shown in Table 2, the MSE and correlation do not vary significantly with different t values, indicating that the choice of t may not have a strong impact on imputation performance within the tested range.

t value	MSE	Correlation
3	0.10	0.96
5	0.12	0.97
10	0.12	0.97
15	0.11	0.95
30	0.08	0.98
50	0.09	0.98

Table 2: Effect of different t values on imputation performance

1.3 How does graph connectivity (different k values) influence denoising quality?

Small k gives a sparser graph with higher variance denoising. but may fragment the graph if too small Larger k gives a denser graph with lower variance denoising. However, too large of a k can lead to oversmoothing and loss of important local structure in the data.

2 Implementing the PHATE Regularized Autoencoder

2.1 Given that cellular data often contains noise, how would you adapt the model into a denoising autoencoder to better recover the underlying signal?

We can corrupt the original data with noise and training the autoencoder to map the corrupted version back to the original, thereby learning robust features and denoising the data.

3 Implementing a Neural ODE to generate trajectories

3.1 Experiment with the Neural ODE density loss by sweeping the k and hinge loss hyper-parameters and briefly report the effects.

As shown in Table 3, increasing the k value generally leads to an increase in density loss. Adjusting the hinge value also affects the density loss, with a hinge value of 0.1 yielding the lowest loss in this experiment.

k value	Hinge Value	Density Loss
5	0.01	0.08
10	0.01	0.10
15	0.01	0.12
5	0.05	0.04
5	0.1	0.02

Table 3: Effect of different k and hinge values on density loss

3.2 What is unbalanced optimal transport, and how would you implement it in this system? Please provide a pseudocode or explanation.

UOT relaxes the mass conservation constraint of traditional OT, allowing for creation or destruction of mass. To implement it, we regularize the balanced OT by replacing the hard constraints with divergence and use Sinkhorn iterations to solve the resulting optimization problem. The python function is `ot.unbalanced.sinkhorn_unbalanced`.

$$\mathcal{L}_{\mathrm{UOT}}(x, y) = \min_{\pi \geq 0} \langle M, \pi \rangle + \tau_1 \, D_{\mathrm{KL}}(\pi \mathbf{1} \,\|\, \mu) + \tau_2 \, D_{\mathrm{KL}}(\pi^\top \mathbf{1} \,\|\, \nu) + \varepsilon \, \mathrm{H}(\pi)$$

Algorithm 1 Unbalanced Optimal Transport Loss: `ot_loss(x, y)`

Require: x, y
Ensure: w_2 (scalar OT cost)
Step 1: Define uniform marginal weights
$\mu \leftarrow \frac{1}{n_x} \cdot \mathbf{1}_{n_x}$
$\nu \leftarrow \frac{1}{n_y} \cdot \mathbf{1}_{n_y}$
Step 2: Compute squared Euclidean ground-cost matrix
$M \leftarrow ||x, y||_2^2$
Step 3: Compute unbalanced Sinkhorn–KL optimal transport
Input parameters: entropic regularization ε, marginal penalty τ
$G \leftarrow \text{SinkhornUnbalanced}(\mu, \nu, M, \varepsilon, \tau)$
Step 4: Compute total transport cost
$w_2 \leftarrow \sum_{i,j} G_{ij} \cdot M_{ij}$
Step 5: Return cost as PyTorch scalar
Output: w_2

3.3 How does the network learn manifold geodesics? Describe the losses and how they promote the computation of manifold geodesics.

The autoencoder maps data to a latent manifold and the neural ODE learns dynamics on this manifold. The neural ODE learns a vector field on the tangent bundle of the manifold. The vector field is contrained by the loss function which penalizes deviations from the geodesic.

The OT loss trains the neural ODE to find a vector field that pushes forward the initial distribution to match the target distribution. The solution to this dynamic OT problem is a set of trajectories that are minimal-energy geodesics under the W_2 metric. By minimizing this loss, the model is penalized for any deviations from this optimal, geodesic-like flow, which encourages efficient, smooth trajectories.

The energy loss directly penalizes the total kinetic energy of the learned transport paths, promoting smoother and more efficient dynamics.

The density loss ensures that the neural ODE's learned dynamics respect the underlying probability distribution of the data, thereby preserving the intrinsic manifold structure.